Analisi di specifiche VHDL

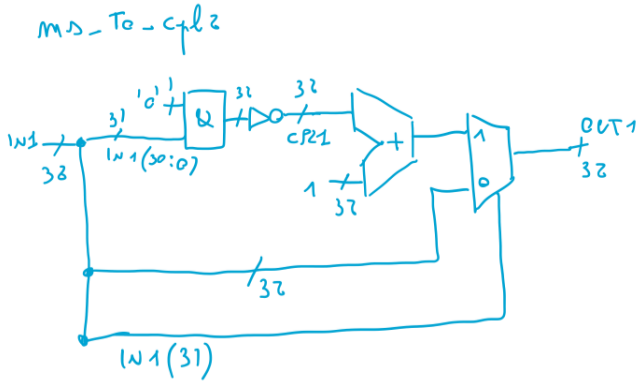
Esercizio 1. Si vuole progettare un modulo combinatorio in grado di eseguire la somma di due numeri in ingresso codificati sia con la notazione complemento a 2 che con quella modulo e segno. Il modulo ha tre segnali di ingresso: A e B a 32 bit che rappresentano i due valori in ingresso ed un segnale CPL2/nMS che specifica se i due valori in ingresso sono codificati con la notazione complemento a 2 (nel caso il segnale è uguale a 1) o modulo e segno (se il segnale assume valore 0). Inoltre il modulo ha due segnali di uscita: RIS a 32 bit che rappresenta il risultato della somma (codificato con la stessa notazione dei valori di ingresso) e OVF che indica se si è verificato un overflow durante l'operazione.

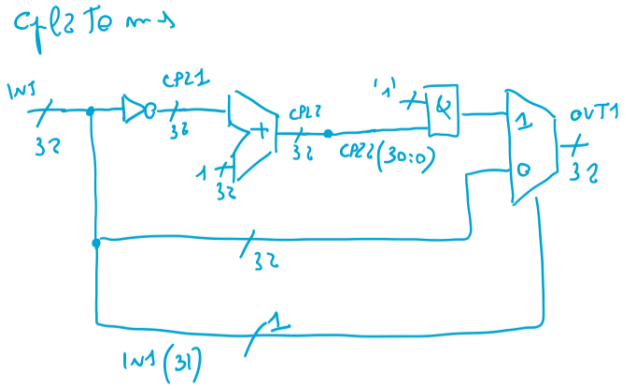
Data la seguente specifica VHDL del modulo descritto, disegnare lo schematico del modulo principale e dei vari sottomoduli.

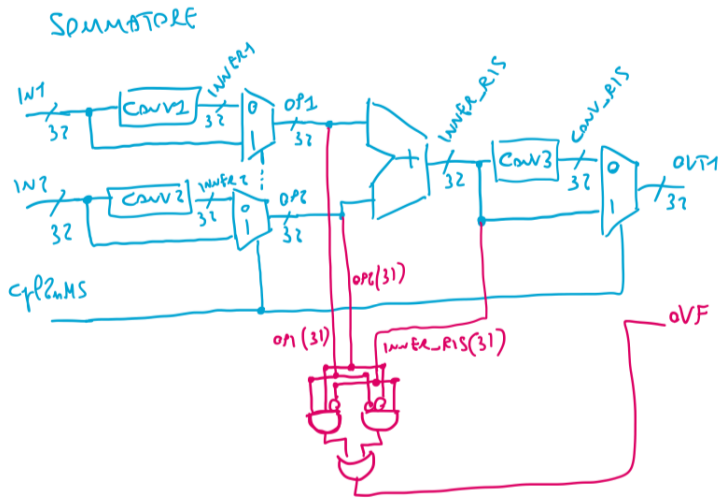
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity ms_to_cpl2 is
  port(
    in1: in std_logic_vector(31 downto 0);
    out1 : out std_logic_vector(31 downto 0)
  );
end ms_to_cpl2;

architecture dataflow of ms_to_cpl2 is
  signal cpl1, inv1 : std_logic_vector(31 downto 0);
begin
  cpl1 <= not ('0' & in1(30 downto 0));
  inv1 <= std_logic_vector(signed(cpl1) + 1);
  out1 <= in1 when in1(31) = '0' else inv1;
end dataflow;

-------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity cpl2_to_ms is
  port(
    in1: in std_logic_vector(31 downto 0);
    out1 : out std_logic_vector(31 downto 0)
  );
end cpl2_to_ms;

architecture dataflow of cpl2_to_ms is
  signal cpl1, cpl2, neg_ms: std_logic_vector(31 downto 0);
begin
  cpl1 <= not in1;
  cpl2 <= std_logic_vector(signed(cpl1) + 1);
  out1 <= in1 when in1(31) = '0' else
          '1' & cpl2(30 downto 0);
end dataflow;

-------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity sommatore is
  port(
    in1, in2: in std_logic_vector(31 downto 0);
    cpl2Nms: in std_logic;
    out1 : out std_logic_vector(31 downto 0);
    ovf : out std_logic
```

```
  );
end sommatore;

architecture mixed of sommatore is
signal inner1, inner2, op1, op2, inner_ris, conv_ris:
          std_logic_vector(31 downto 0);

component ms_to_cpl2 is
  port(
    in1: in std_logic_vector(31 downto 0);
    out1 : out std_logic_vector(31 downto 0)
  );
end component;

component cpl2_to_ms is
  port(
    in1: in std_logic_vector(31 downto 0);
    out1 : out std_logic_vector(31 downto 0)
);
end component;

begin
  conv1: ms_to_cpl2
    port map(in1, inner1);

  conv2: ms_to_cpl2
    port map(in2, inner2);

  op1 <= in1 when cpl2Nms ='1' else
         inner1;

  op2 <= in2 when cpl2Nms ='1' else
         inner2;

  inner_ris <= std_logic_vector(SIGNED(op1) + SIGNED(op2));

  ovf <= (op1(31) and op2(31) and not inner_ris(31)) or
             (not op1(31) and not op2(31) and inner_ris(31));

  conv3: cpl2_to_ms
    port map(inner_ris, conv_ris);

  out1 <= inner_ris when cpl2Nms ='1' else
         conv_ris;
end mixed;
```

SOLUZIONE:


ms_To_cpl2
IN1
32
31
'0'
Q
32
32
CPL1
IN1(30:0)
1
32
1
0
OUT1
32
32
IN1(31)


cpl2 To ms
IN1
32
CPL1
32
CPL2
32
'1'
Q
CPL2(30:0)
1
32
1
0
OUT1
32
32
IN1(31)


SOMMATORE
IN1
32
CONV1
INVER1
32
OP1
32
IN2
32
CONV2
INVER2
32
OP2
32
INVER_RIS
32
CONV3
CONV_RIS
32
OUT1
32
cpl2MS
OP2(31)
OP1(31)
INVER_RIS(31)
OVF

Esercizio 2. Si vuole progettare un modulo ADDSUB con tre ingressi A, B (a 4 bit) e notAdd_sub (ad 1 bit) e due uscite C (a 4 bit) e OVF (ad 1 bit). Se notAdd_sub=0, il modulo esegue la somma in complemento a 2 dei due numeri in ingresso, in alternativa la sottrazione e presenta il risultato sull'uscita C; su OVF viene riportato l'eventuale overflow.

Data la seguente specifica VHDL del modulo descritto, disegnare lo schematico del modulo principale e dei vari sottomoduli.

```
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
  port (x, y : in std_logic;
        s, c : out std_logic);
end half_adder;

architecture dataflow of half_adder is
begin
  s <= x xor y;
  c <= x and y;
end dataflow;

----------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
  port (x, y, z : in std_logic;
        s, c : out std_logic);
end full_adder;

architecture structural of full_adder is
  component half_adder
    port(x, y : in std_logic;
         s, c : out std_logic);
  end component;

  signal hs, hc, tc: std_logic;

begin
  HA1: half_adder
     port map (x, y, hs, hc);
  HA2: half_adder
     port map (hs, z, s, tc);
  c <= tc or hc;
end structural;

----------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity adder_4 is
  port(A, B : in  std_logic_vector(3 downto 0);
       C0: in std_logic;
       S : out std_logic_vector(3 downto 0);
       C4: out std_logic);
end adder_4;

architecture structural of adder_4 is
  component full_adder is
    port(x, y, z : in std_logic;
         s, c : out std_logic);
  end component;

  signal C: std_logic_vector(4 downto 0);
```

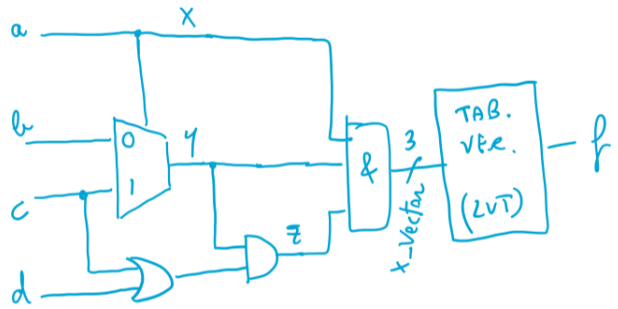
```
begin
  Bit0: full_adder
    port map (A(0), B(0), C(0), S(0), C(1));
  Bit1: full_adder
    port map (A(1), B(1), C(1), S(1), C(2));
  Bit2: full_adder
    port map (A(2), B(2), C(2), S(2), C(3));
  Bit3: full_adder
    port map (A(3), B(3), C(3), S(3), C(4));
  C(0) <= C0;
  C4 <= C(4);
end structural;

----------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity addsub_4 is
  port(A, B : in  std_logic_vector(3 downto 0);
       notAdd_sub : in std_logic;
       RIS : out std_logic_vector(3 downto 0);
       OVF: out std_logic);
end addsub_4;

architecture impl of addsub_4 is
  component adder_4 is
    port(A, B : in  std_logic_vector(3 downto 0);
         C0: in std_logic;
         S : out std_logic_vector(3 downto 0);
         C4: out std_logic);
  end component;

  signal internalB, internalRIS, internalAdd_notSub: std_logic_vector(3 downto 0);
  signal pos_ovf, neg_ovf: std_logic;

begin
  internalB <= B XOR (notAdd_sub & notAdd_sub & notAdd_sub & notAdd_sub);

  adder1: adder_4
    port map (A=>A,B=>internalB,C0=>notAdd_sub,S=>internalRIS,C4=>open);

  OVF <= (A(3) and internalB(3) and not internalRIS(3)) or
         (not A(3) and not internalB(3) and internalRIS(3));

  RIS<=internalRIS;
end impl;
```

SOLUZIONE: TODO

Esercizio 3. Disegnare il circuito descritto dal seguente codice VHDL e derivarne la tabella delle verità.

```
library ieee;
use ieee.std_logic_1164.all;

entity myComponent is
  port(  a : in  std_logic;
         b : in  std_logic;
         c : in  std_logic;
         d : in  std_logic;
         f : out  std_logic;
  );
end entity;

architecture rtl of myComponent is
  signal x, y, z: std_logic;
  signal x_vector: std_logic_vector(2 downto 0)
begin
  x <= a;
  z <= (c or d) and y;
  x_vector <= x & y & z;

  process (x_vector)
  begin
    case x_vector
      when "000"  => f <= '1';
      when "001"  => f <= '1';
      when "010"  => f <= '1';
      when "100"  => f <= '1';
      when others => f <= '0';
    end case;
  end process;

  process (a, b, c)
  begin
    if a = '0' then
       y <= b;
    else
       y <= c;
    end if;
  end process;
end rtl;
```

SOLUZIONE:



a	b	c	d	x	y	z	f
0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	1
0	0	1	1	0	0	0	1
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	0
1	0	0	0	1	0	0	1
1	0	0	1	1	0	0	1
1	0	1	0	1	1	1	0
1	0	1	1	1	1	1	0
1	1	0	0	1	0	0	1
1	1	0	1	1	0	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	0

Esercizio 4. Data la seguente specifica VHDL di una rete multilivello:

```
library ieee;
use ieee.std_logic_1164.all;

entity multi_level_PAL is
  port( x        : in   std_logic;
    y    : in   std_logic;
    z    : in   std_logic;
    t    : in   std_logic;
    f    : out  std_logic;
    g    : out  std_logic; );
end entity;

architecture rtl of multi_level_PAL is
  signal p,q,r,s : std_logic;
begin
  p <= x or ((not y) and z);
  q <= ((p and (not z)) or (x and (not y))) or (x and z);
  r <= p or (not x);
  s <= z or (not y);
  f <= ((not q) and r) or ((not x) and z) or ((not y) and t) or (t and z);
  g <= (r and s) or ((not z) and t);
end rtl;
```

1. Si rappresenti il grafo multilivello equivalente a tale specifica.
2. Applicare alla rete multilivello le seguenti trasformazioni e disegnare (o riscrivere le equazioni di tutti i nodi) dopo ogni Passo (1, 2 e 3) la rete trasformata.
   (a) Eliminate(p); Simplify(q) Simplify(r); Cost()
   (b) Eliminate(q); Eliminate(r); Simplify(f); Simplify(g); Cost()
   (c) Factor(f); Substitute(s); Cost()
3. Realizzare la rete risultante con una logica programmabile PAL con OR a due ingressi e con una PLA.

Si ricorda che:

- SIMPLIFY (x): Minimizzazione a due livelli del nodo x nella forma somma di prodotti.
- ELIMINATE (x): Eliminazione del nodo x non vincolata ad una condizione sul costo. Il nodo x viene eliminato e la sua espressione viene riportata nei nodi a valle.
- COST(): Calcolare il costo della rete in termini di letterali.
- FACTOR(x): Fattorizzazione del nodo x.
- SUBSTITUTE(x): Verifica per ciascun altro nodo se x e un divisore o un fattore ed in caso positivo esprimere tale nodo in funzione di x.

SOLUZIONE: TODO

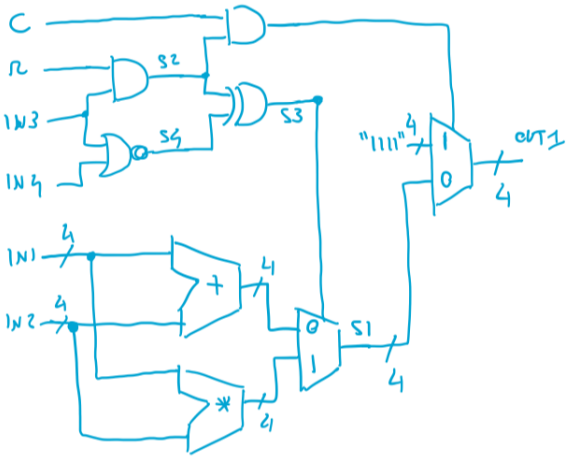
Esercizio 5. Data la seguente specifica VHDL:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all

entity exam1 is
  port(
    in1, in2 : in std_logic_vector(3 DOWNTO 0);
    in3, in4 : in std_logic;
    c, r : in std_logic;
    out1 : out std_logic_vector(3 DOWNTO 0)
  );
end exam1;

architecture mixed of exam1 is
  signal s2,s3,s4 : std_logic;
  signal s1 : std_logic_vector(3 DOWNTO 0);
begin
  PROC1 : process (c, s1, s2)
  begin
    if(c='1' and s2='1') then
      out1 <= "1111";
    else
      out1 <= s1;
    end if;
  end process;

  PROC2: process(in1, in2, s3)
  begin
    if ( s3='0' ) then
      s1 <= std_logic_vector(SIGNED(in1) + SIGNED(in2));
    else
      s1 <= std_logic_vector(SIGNED(in1) * SIGNED(in2));
  end process;

  s2 <= r and in3;
  s3 <= s4 xor s2;
  s4 <= not (in3 or in4);
end mixed;
```

1. Indicare se il circuito descritto è un circuito combinatorio o sequenziale con le opportune motivazioni.
2. Disegnare un circuito composto da componenti elementari (porte logiche, multiplexer, bistabili) che implementi il modello VHDL proposto.

SOLUZIONE:

LA SPECIFICA È COMBINATORIA POICHÉ NON SONO PRESENTI ELEMENTI DI MEMORIA



Esercizio 6. Dire che tipo di circuito è descritto dal seguente sorgente VHDL. Eseguire la sintesi ottima del circuito mediante gli approcci visti a lezione.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mistero is
  port(
      clk:  in std_logic;
      rst:  in std_logic;
      o:  out std_logic_vector(2 downto 0)
  );
end mistero;

architecture mistero_arch of mistero is
  signal next_s, current_state: std_logic_vector(2 downto 0);
begin
  state_reg: process(clk, rst)
  begin
    if rst='1' then
      current_state <= "000";
    elsif rising_edge(clk) then
      current_state <= next_s;
    end if;
  end process;

  foo: process(current_state)
  begin
    case current_state is
      when "000" =>
        next_s <= "001";
      when "001" =>
        next_s <= "011";
      when "011" =>
        next_s <= "111";
      when "111" =>
        next_s <= "110";
      when "110" =>
        next_s <= "100";
      when "100" =>
        next_s <= "000";
      when others =>
        next_s <= "---";
    end case;
  end process;

  o <= current_state;
end mistero_arch;
```

SOLUZIONE:

E' UN CONTATORE (USIAMO FF_D)

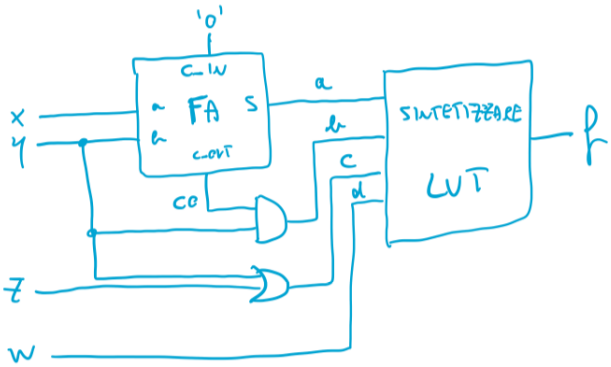
	TAB. TRANS.			TAB. ECC.		
Q_2 Q_1 Q_0	Q_2^*	Q_1^*	Q_0^*	D_2	D_1	D_0
0 0 0	0	0	1	0	0	1
0 0 1	0	1	1	0	1	1
0 1 0	–	–	–	–	–	–
0 1 1	1	1	1	1	1	1
1 0 0	0	0	0	0	0	0
1 0 1	–	–	–	–	–	–
1 1 0	1	0	0	1	0	0
1 1 1	1	1	0	1	1	0

Esercizio 7. Data la seguente specifica VHDL:

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all

entity exam1 is
  port(
    x, y, z, w: in std_logic;
    f: out std_logic
  );
end exam1;

architecture mixed of exam1 is
  signal a,b,c,d, co : std_logic;

  -- questo componente e' un full adder
  component fulladder is
    port(
      a, b, c_in: in std_logic;
      s, c_out: out std_logic
    );
   end component;

begin
  fa: fulladder
  port map(x,y,'0',a,co);

  b <= co and y;
  c <= y or z;
  d <= w;

  sintetizzare: process(a,b,c,d)
    variable value;
  begin
    value := a & b & c & d;
    case value is
      when "0001" | "0010" | "1000" |
           "1010" | "1101" | "1110" =>
        f <= '1';
      when others =>
        f <= '0';
  end process;
end mixed;
```

1. Indicare se il circuito descritto è un circuito combinatorio o sequenziale con le opportune motivazioni.
2. Disegnare un circuito composto da componenti elementari (porte logiche, multiplexer, bistabili) che implementi il modello VHDL proposto. Nel disegno illustrare il processo `sintetizzare` come un rettangolo mostrando opportunamente gli ingressi e le uscite
3. Sintetizzare la funzione $f(a,b,c,d)$ descritta dal processo `sintetizzare` mediante il metodo delle mappe di Karnaugh mostrando tutti i passaggi fino ad arrivare all'equazione della soluzione (generazione di tutti gli implicanti primi, individuazione della copertura, calcolo del costo).
4. Riscrivere la funzione $f(a,b,c,d)$ sfruttando eventuali condizioni di indifferenza dovute dal circuito a monte. Rieseguire la sintesi della nuova funzione $f'(a,b,c,d)$ ottenuta arrivando all'equazione finale.
5. Confrontare le due soluzioni ottenute stabilendo quale costa meno e commentando brevemente il motivo.

SOLUZIONE:



SINTETIZZARE: $f(a,b,c,d) : ON(1,2,8,10,13,14)$

ab \ cd	00	01	11	10
00	0	1	0	1
01	0	0	0	0
11	0	1	0	1
10	1	0	0	1

$A = \bar{a}\bar{b}\bar{c}d$

$B = \bar{b}c\bar{d}$

$C = ab\bar{c}d$

$D = ac\bar{d}$

$E = a\bar{b}\bar{d}$

$$f = \bar{a}\bar{b}\bar{c}d + \bar{b}c\bar{d} + ab\bar{c}d + ac\bar{d} + a\bar{b}\bar{d}$$

CALCOLO DEI DON'T CARE

x	y	z	w	a	b	c	d	
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	0	2
0	0	1	1	0	0	1	1	3
0	1	0	0	1	0	1	0	10
0	1	0	1	1	0	1	1	11
0	1	1	0	1	0	1	0	10
0	1	1	1	1	0	1	1	11
1	0	0	0	1	0	0	0	8
1	0	0	1	1	0	0	1	9
1	0	1	0	1	0	1	0	10
1	0	1	1	1	0	1	1	11
1	1	0	0	0	1	1	0	6
1	1	0	1	0	1	1	1	7
1	1	1	0	0	1	1	0	6
1	1	1	1	0	1	1	1	7

SEQUENZE CONTROLLABILI: $\{0, 1, 2, 3, 6, 7, 8, 9, 10, 11\}$

$DC = \{0, \dots, 15\} /$ SEQ. CONTROLLABILI:

$\{4, 5, 12, 13, 14, 15\}$

$f^* = ON(1, 2, 8, 10)\ DC(4, 5, 12, 13, 14, 15)$

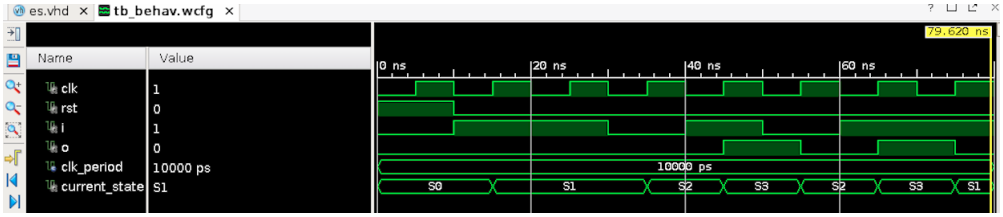
RIESEGUIRE SINTESI...

Esercizio 8. Data la seguente specifica VHDL. Disegnare le forme d'onda dei segnali in ingresso ed uscita all'istanza `fsm1` del componente `fsm`.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fsm is
  port(
      i:  in std_logic;
      clk:  in std_logic;
      rst:  in std_logic;
      o:  out std_logic
  );
end fsm;

architecture fsm_arch of fsm is
  type state_type is (S0, S1, S2, S3);
  signal next_state, current_state:

begin
  state_reg: process(clk, rst)
  begin
    if rst='1' then
      current_state <= S0;
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

  delta_lambda: process(current_state, i)
  begin
    case current_state is
      when S0 =>
        if i='0' then
          next_state <= S0;
        else
          next_state <= S1;
        end if;
        o <= '0';
      when S1 =>
        if i='0' then
          next_state <= S2;
        else
          next_state <= S1;
        end if;
        o <= '0';
      when S2 =>
        if i='0' then
          next_state <= S0;
        else
          next_state <= S3;
        end if;
        o <= '0';
      when S3 =>
        if i='0' then
          next_state <= S2;
        else
          next_state <= S1;
        end if;
        o <= '1';
    end case;
  end process;
end fsm_arch;
```

```vhdl
library IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_TEXTIO.ALL;
USE STD.TEXTIO.ALL;

ENTITY tb IS
END tb;

ARCHITECTURE testbench_arch OF tb IS
  COMPONENT fsm IS
    port(
      i:  in std_logic;
      clk:  in std_logic;
      rst:  in std_logic;
      o:  out std_logic
);
state_type;
  END COMPONENT;

  CONSTANT clk_period : time := 10 ns;

  SIGNAL clk : std_logic := '0';
  SIGNAL rst : std_logic := '1';
  SIGNAL i : std_logic := '0';
  SIGNAL o : std_logic := '0';

BEGIN

  fsm1: fsm
    PORT MAP(i, clk, rst, o);

  clk_process: PROCESS
  BEGIN
    clk <= '0';
    WAIT FOR clk_period/2;
    clk <= '1';
    WAIT FOR clk_period/2;
  END PROCESS;

  stimula_process: PROCESS
  BEGIN
    WAIT FOR clk_period;
    rst <= '0';
    i <= '1';
    WAIT FOR clk_period;
    i <= '1';
    WAIT FOR clk_period;
    i <= '0';
    WAIT FOR clk_period;
    i <= '1';
    WAIT FOR clk_period;
    i <= '0';
    WAIT FOR clk_period;
    i <= '1';
    WAIT FOR clk_period;
    i <= '1';
    WAIT FOR clk_period;
    ASSERT(FALSE) REPORT "Simulation␣OK." SEVERITY FAILURE;
  END PROCESS;

END testbench_arch;
```

SOLUZIONE:


es.vhd
tb_behav.wcfg
Name
Value
clk
1
rst
0
i
1
o
0
clk_period
10000 ps
current_state
S1
79,620 ns
0 ns
20 ns
40 ns
60 ns
10000 ps
S0
S1
S2
S3
S2
S3
S1